Data Gathering Service

In the previous design, the trie was updated every time a user typed a search query.

This real-time update approach is not practical for large-scale systems because:

1. Users may generate billions of queries daily.
2. Updating the trie on every query is computationally expensive.
3. Frequent trie updates slow down the query service.
4. Top autocomplete suggestions usually do not change rapidly.

Therefore, rebuilding the trie continuously is unnecessary for many applications.

Key Idea

To design a scalable system, we separate:

- Query collection
- Data aggregation
- Trie building
- Query serving

Instead of updating the trie in real time, search queries are collected and processed periodically.

Real-Time vs Batch Processing

Different applications require different update frequencies.

Application	Update Frequency
Twitter/X	Near real-time

Google Search	Daily/Weekly updates may be sufficient

For this design, we assume:

- Trie is rebuilt weekly.

Redesigned Data Gathering Service

Figure shows the improved architecture.

The redesigned data gathering service contains:

1. Analytics Logs
2. Aggregators
3. Aggregated Data
4. Workers
5. Trie Cache
6. Trie DB

1. Analytics Logs

Analytics logs store raw search query data.

Characteristics:

- Append-only
- Large volume
- Not indexed
- Optimized for writes

Example Analytics Log Table

Query	Time
tree	2026-05-01
try	2026-05-01
tree	2026-05-02
true	2026-05-03

Each time a user performs a search, a log entry is appended.

Why Logs Are Used

Advantages:

- Very high write throughput
- Easy to scale
- Good for analytics pipelines

- Suitable for distributed systems

2. Aggregators

Raw analytics logs are too large and unsuitable for direct trie construction.

Aggregators process raw logs and generate summarized data.

Responsibilities:

- Count query frequencies
- Merge duplicate entries
- Produce aggregated statistics

Aggregation Frequency

Aggregation interval depends on the application.

Real-time systems

Examples:

- Twitter trends
- Live hashtags

Use:

- Minute-level or hourly aggregation

Non-real-time systems

Examples:

- Google autocomplete

Use:

- Daily or weekly aggregation

In this design:

- Data is aggregated weekly.

3. Aggregated Data

After aggregation, data is transformed into a compact format.

Example Aggregated Weekly Data

Query	Time	Frequency
tree	Week 1	1200
try	Week 1	980
true	Week 1	1500

Explanation:

- `time` → start time of aggregation period
- `frequency` → total searches during that week

This aggregated data is later used to build the trie.

4. Workers

Workers are background servers that perform asynchronous jobs.

Responsibilities:

- Read aggregated data
- Build trie structures
- Generate top-k caches
- Store trie into persistent storage

Workers run periodically, such as:

- Daily
- Weekly

In this design:

- Workers rebuild the trie weekly.

5. Trie Cache

Trie cache is an in-memory distributed cache system.

Purpose:

- Fast query lookup
- Low latency autocomplete response

Characteristics:

- Stores trie in memory
- Periodically refreshed
- Uses snapshots from Trie DB

Benefits:

- Extremely fast reads
- Reduced database load

6. Trie DB

Trie DB is the persistent storage layer for trie data.

Two approaches are commonly used.

Option 1: Document Store

Examples:

- MongoDB
- Couchbase

Approach:

1. Build trie periodically
2. Serialize trie structure
3. Store serialized snapshot in DB

Why document stores work well:

- Trie is hierarchical
- Serialized trie fits naturally into documents

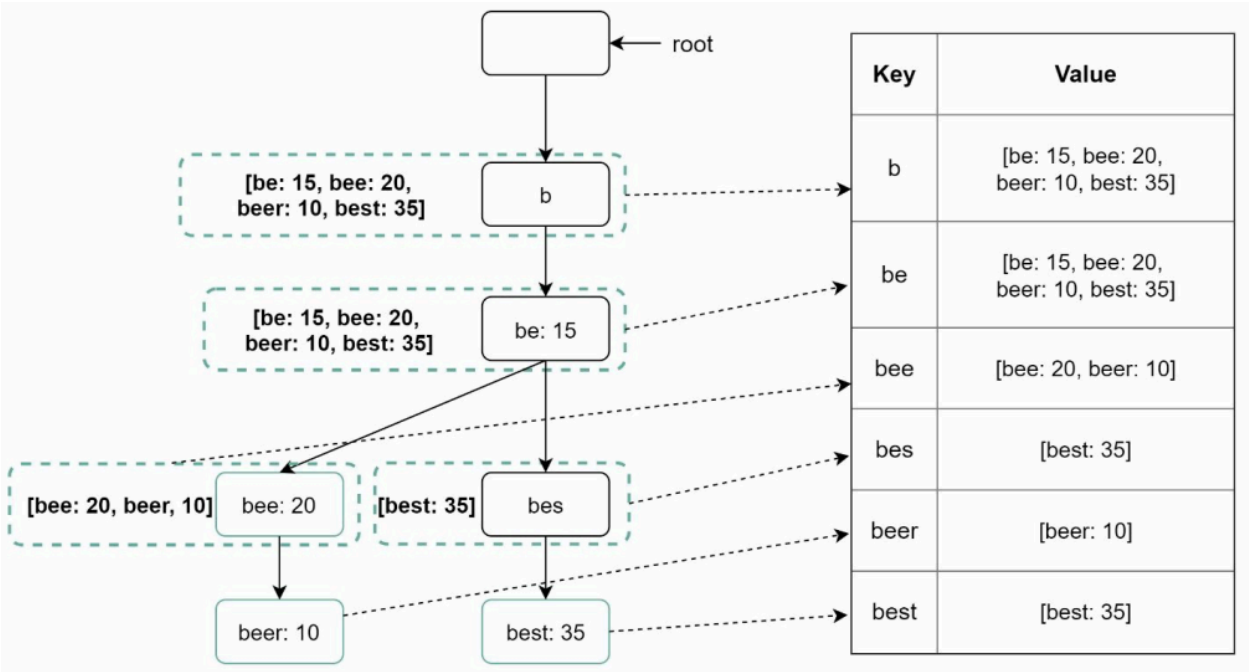
Option 2: Key-Value Store

Another approach is representing trie as hash table entries.

Logic:

- Every prefix becomes a key
- Trie node data becomes the value

Trie to Hash Table Mapping



Example:

Key	Value
t	top queries for prefix t
tr	top queries for prefix tr
tre	top queries for prefix tre

Each trie node maps to:

```
None
<prefix, node_data>
```

where:

- key = prefix
- value = frequency info + top queries

Advantages of Key-Value Store

Benefits:

- Very fast lookups
- Easy horizontal scaling
- Efficient distributed storage
- Simple prefix mapping

Overall Workflow

Step 1

Users submit search queries.

Step 2

Queries are appended to analytics logs.

Step 3

Aggregators process logs periodically.

Step 4

Workers build optimized trie structures.

Step 5

Trie is stored in Trie DB.

Step 6

Trie snapshots are loaded into Trie Cache.

Step 7

Query service retrieves autocomplete suggestions from cache.

Benefits of the Redesigned Architecture

Improvement	Benefit
Batch aggregation	Reduced computation
Trie rebuilding	Better scalability
Trie cache	Faster responses
Workers	Asynchronous processing
Persistent Trie DB	Durability
Key-value mapping	Efficient retrieval

Summary

The redesigned data gathering service improves scalability by:

- Separating query ingestion from trie construction
- Using batch aggregation instead of real-time updates
- Storing trie snapshots efficiently
- Serving autocomplete queries directly from memory cache

This architecture supports:

- High throughput
- Low latency
- Horizontal scalability
- Efficient autocomplete retrieval at large scale.